DA5400 (July-Nov 2025)
Assignment 2 Report
Nishchala Mukku (EE24S004)

Q1:

Refer to **Q1.m** file.
You are given a data-set with 400 data points in $\{0,1\}^{50}$ generated from a mixture of some distribution in the file A2Q1.csv.
(**Hint:** Each datapoint is a flattened version of a $\{0,1\}^{10\times5}$ matrix.)

(i) **Determine which probabilisitic mixture could have generated this data (It is not a Gaussian mixture). Derive the EM algorithm for your choice of mixture and show your calculations. Write a piece of code to implement the algorithm you derived by setting the number of mixtures $K = 4$. Plot the log-likelihood (averaged over 100 random initializations) as a function of iterations.**
Sol: The dataset X contains binary features (values 0 or 1). A Gaussian mixture assumes continuous data and can produce invalid intermediate values (e.g., $x_j = 0.3$) not possible in binary form. Hence, a **Mixture of Bernoulli Distributions** correctly models binary data by defining probabilities over $\{0,1\}$ for each feature dimension.

The dataset is $X = \{x_n\}_{n=1}^{N}$, where each data vector $x_n \in \{0,1\}^D$. Assume there are K latent mixture components (clusters) indicated by a hidden variable $z_n \in \{1,\dots,K\}$.

The model parameters are:

$$\Theta = \{\pi_k, \mu_k\}_{k=1}^{K}$$

where $\pi_k \geq 0$ and $\sum_{k=1}^{K} \pi_k = 1$. Each $\mu_k = (\mu_{k1},\dots,\mu_{kD})$ with $\mu_{kj} \in (0,1)$ representing the Bernoulli probability that feature j is active (equals 1) in cluster k.

Given the latent variable z_n, the conditional likelihood for data point x_n is:

$$p(x_n \mid z_n = k, \mu_k) = \prod_{j=1}^{D} \mu_{kj}^{x_{nj}} (1-\mu_{kj})^{1-x_{nj}}.$$

The marginal (observed) likelihood is:

$$p(x_n \mid \Theta) = \sum_{k=1}^{K} \pi_k \, p(x_n \mid z_n = k, \mu_k).$$

The log-likelihood of the full dataset is:

$$\mathcal{L}(\Theta) = \sum_{n=1}^{N} \log \Big(\sum_{k=1}^{K} \pi_k \prod_{j=1}^{D} \mu_{kj}^{x_{nj}} (1-\mu_{kj})^{1-x_{nj}} \Big).$$

E-step

Compute the posterior responsibility:

$$r_{nk} = P(z_n = k \mid \mathbf{x}_n, \Theta^{(t)}) = \frac{\pi_k \prod_{j=1}^{D} \mu_{kj}^{x_{nj}} (1-\mu_{kj})^{1-x_{nj}}}{\sum_{\ell=1}^{K} \pi_\ell \prod_{j=1}^{D} \mu_{\ell j}^{x_{nj}} (1-\mu_{\ell j})^{1-x_{nj}}}.$$

Taking log,

$$\ell_{nk} = \log \pi_k + \sum_{j=1}^{D} \big[x_{nj} \log \mu_{kj} + (1-x_{nj}) \log(1-\mu_{kj}) \big],$$

$$\log p(\mathbf{x}_n) = m_n + \log \sum_k \exp(\ell_{nk} - m_n), \quad m_n = \max_k \ell_{nk},$$

$$r_{nk} = \exp(\ell_{nk} - \log p(\mathbf{x}_n)).$$

M-step

Maximize the expected complete-data log-likelihood:

$$Q(\Theta; \Theta^{(t)}) = \sum_{n,k} r_{nk} \Big[\log \pi_k + \sum_{j=1}^{D} \big(x_{nj} \log \mu_{kj} + (1-x_{nj}) \log(1-\mu_{kj}) \big) \Big].$$

Parameter updates:

$$N_k = \sum_{n=1}^{N} r_{nk}, \qquad \pi_k = \frac{N_k}{N}, \qquad \mu_{kj} = \frac{\sum_{n=1}^{N} r_{nk} x_{nj}}{N_k}.$$

The observed log-likelihood:

$$\mathcal{L}^{(t)} = \sum_{n=1}^{N} \log p(\mathbf{x}_n) = \sum_{n=1}^{N} \Big[m_n + \log \sum_k \exp(\ell_{nk} - m_n) \Big],$$

monotonically increases with each EM iteration. Convergence occurs when

$$|\mathcal{L}^{(t+1)} - \mathcal{L}^{(t)}| < \varepsilon.$$

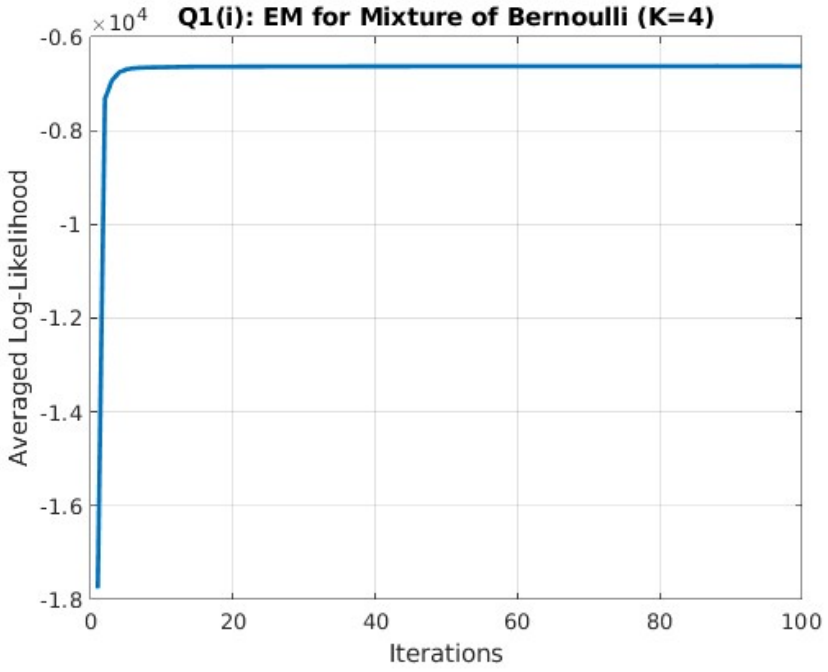

Figure 1: EM Algorithm for Bernoulli

$$\boxed{\begin{aligned} r_{nk} &= \frac{\pi_k \prod_j \mu_{kj}^{x_{nj}} (1-\mu_{kj})^{1-x_{nj}}}{\sum_\ell \pi_\ell \prod_j \mu_{\ell j}^{x_{nj}} (1-\mu_{\ell j})^{1-x_{nj}}}, \\ N_k &= \sum_n r_{nk}, \quad \pi_k = \frac{N_k}{N}, \quad \mu_{kj} = \frac{\sum_n r_{nk} x_{nj}}{N_k}. \end{aligned}}$$

Now, the EM algorithm is run for 100 iterations and K=4. We get the following plot as in fig.1. The plot shows the averaged log-likelihood converging extremely quickly (in about 10 iterations) to a stable value of approximately -0.65 x 10^4. This suggests the model is good, natural fit for the data structure. As expected the log-likelihood is negative, since it is log of probabilities which are non negative quanitites.

(ii) **Assume that the same data was infact generated from a mixture of Gaussians with 4 mixtures. Implement the EM algorithm and plot the log-likelihood (averaged over 100 random initializations of the parameters) as a function of iterations. How does the plot compare with the plot from part (i)? Provide insights that you draw from this experiment.**
Sol: Let $\mathbf{x}_1, \ldots, \mathbf{x}_N \in \mathbb{R}^D$ be observed vectors. Assume a mixture of K

Gaussians with parameters

$$\Theta = \{\pi_k, \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k\}_{k=1}^{K}, \qquad \pi_k \geq 0,\ \sum_{k=1}^{K} \pi_k = 1,$$

where component density

$$p(\mathbf{x} \mid k) = \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) = \frac{1}{(2\pi)^{D/2}|\boldsymbol{\Sigma}_k|^{1/2}} \exp\Big(-\tfrac{1}{2}(\mathbf{x}-\boldsymbol{\mu}_k)^\top \boldsymbol{\Sigma}_k^{-1}(\mathbf{x}-\boldsymbol{\mu}_k)\Big).$$

Introduce latent one-hot indicators $z_n \in \{1, \ldots, K\}$. Complete-data joint:

$$p(\mathbf{x}_n, z_n = k \mid \Theta) = \pi_k \, \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k).$$

Complete-data log-likelihood:

$$\log p(X, Z \mid \Theta) = \sum_{n=1}^{N} \sum_{k=1}^{K} \mathbb{I}[z_n = k]\Big(\log \pi_k + \log \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)\Big).$$

E-step

Now posterior,

$$\gamma_{nk} \equiv P(z_n = k \mid \mathbf{x}_n, \Theta^{(t)}) = \frac{\pi_k^{(t)} \, \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k^{(t)}, \boldsymbol{\Sigma}_k^{(t)})}{\sum_{\ell=1}^{K} \pi_\ell^{(t)} \, \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_\ell^{(t)}, \boldsymbol{\Sigma}_\ell^{(t)})}.$$

This is Bayes' rule applied to mixture components and computing log-densities

$$\ell_{nk} = \log \pi_k^{(t)} + \log \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k^{(t)}, \boldsymbol{\Sigma}_k^{(t)}),$$

use the log-sum-exp trick then exponentiate differences to obtain γ_{nk}.

$$Q(\Theta; \Theta^{(t)}) = \mathbb{E}_{Z|X,\Theta^{(t)}}[\log p(X, Z \mid \Theta)] = \sum_{n=1}^{N} \sum_{k=1}^{K} \gamma_{nk}\Big(\log \pi_k + \log \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)\Big).$$

M-step

Define

$$N_k = \sum_{n=1}^{N} \gamma_{nk}.$$

Update for mixing weights π_k. Maximize $\sum_k N_k \log \pi_k$ subject to $\sum_k \pi_k = 1$. Using Lagrange multiplier λ:

$$\mathcal{L} = \sum_k N_k \log \pi_k + \lambda\Big(1 - \sum_k \pi_k\Big).$$

Set derivative zero:

$$\frac{N_k}{\pi_k} - \lambda = 0 \Rightarrow \pi_k = \frac{N_k}{\lambda}.$$

Enforce constraint $\sum_k \pi_k = 1$ gives $\lambda = N$. Thus

$$\boxed{\pi_k^{\text{new}} = \frac{N_k}{N}.}$$

Update for means $\boldsymbol{\mu}_k$. Consider the terms in Q depending on $\boldsymbol{\mu}_k$:

$$\sum_{n=1}^{N} \gamma_{nk}\Big(-\tfrac{1}{2}(\mathbf{x}_n - \boldsymbol{\mu}_k)^\top \boldsymbol{\Sigma}_k^{-1}(\mathbf{x}_n - \boldsymbol{\mu}_k)\Big).$$

Differentiate w.r.t. $\boldsymbol{\mu}_k$ and set to zero:

$$\sum_{n=1}^{N} \gamma_{nk} \boldsymbol{\Sigma}_k^{-1}(\mathbf{x}_n - \boldsymbol{\mu}_k) = 0 \Rightarrow \boldsymbol{\mu}_k^{\text{new}} = \frac{1}{N_k}\sum_{n=1}^{N} \gamma_{nk}\mathbf{x}_n.$$

Update for covariances $\boldsymbol{\Sigma}_k$. Collect covariance terms:

$$Q_\Sigma = -\tfrac{1}{2}\sum_{n=1}^{N} \gamma_{nk}\Big[\log|\boldsymbol{\Sigma}_k| + (\mathbf{x}_n - \boldsymbol{\mu}_k)^\top \boldsymbol{\Sigma}_k^{-1}(\mathbf{x}_n - \boldsymbol{\mu}_k)\Big].$$

Differentiate w.r.t. $\boldsymbol{\Sigma}_k^{-1}$ or use matrix derivatives. Setting derivative zero yields

$$\boldsymbol{\Sigma}_k^{\text{new}} = \frac{1}{N_k}\sum_{n=1}^{N} \gamma_{nk}(\mathbf{x}_n - \boldsymbol{\mu}_k^{\text{new}})(\mathbf{x}_n - \boldsymbol{\mu}_k^{\text{new}})^\top.$$

This is the weighted sample covariance for component k. If numerical regularization is needed add ϵI.

$$\gamma_{nk} = \frac{\pi_k \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_{\ell=1}^{K} \pi_\ell \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_\ell, \boldsymbol{\Sigma}_\ell)},$$

$$N_k = \sum_{n=1}^{N} \gamma_{nk},$$

$$\pi_k^{\text{new}} = \frac{N_k}{N},$$

$$\boldsymbol{\mu}_k^{\text{new}} = \frac{1}{N_k} \sum_{n=1}^{N} \gamma_{nk} \mathbf{x}_n,$$

$$\boldsymbol{\Sigma}_k^{\text{new}} = \frac{1}{N_k} \sum_{n=1}^{N} \gamma_{nk} (\mathbf{x}_n - \boldsymbol{\mu}_k^{\text{new}})(\mathbf{x}_n - \boldsymbol{\mu}_k^{\text{new}})^\top.$$

Observed log-likelihood:

$$\mathcal{L}(\Theta) = \sum_{n=1}^{N} \log \Big(\sum_{k=1}^{K} \pi_k \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \Big).$$

EM guarantees non-decreasing $\mathcal{L}$ at each iteration. Stop when $|\mathcal{L}^{(t+1)} - \mathcal{L}^{(t)}| < \varepsilon$.

Now, the EM algorithm is run for 100 iterations just like previously and we get the following plot fig.2. The Gaussian Mixture Model (GMM) converges to a final log-likelihood of about 0.4 x 10^4, while the Bernoulli Mixture Model (BMM) converges to -0.65 x 10^4. The GMM log-likelihood is numerically higher than the BMM log-likelihood. A Gaussian model uses a probability density function (PDF), which can be greater than 1 (e.g., a very skinny, tall bell curve). This allows its log-likelihood to be positive. We cannot compare the likelihoods, higher value, doesn't mean better model.

This is the worst choice. It is theoretically inappropriate. It models continuous data, but our data is discrete and binary. The resulting high log-likelihood is a misleading numerical artifact, not a sign of a good fit.

(iii) **Run the K-means algorithm with $K = 4$ on the same data. Plot the objective of K-means as a function of iterations.**
Sol: K-Means can be viewed as a limiting case of the Expectation-Maximization (EM) algorithm applied to a Gaussian Mixture Model (GMM) with the following constraints:

$$\Sigma_k = \sigma^2 I, \quad \sigma^2 \to 0, \quad \pi_k = \frac{1}{K}$$

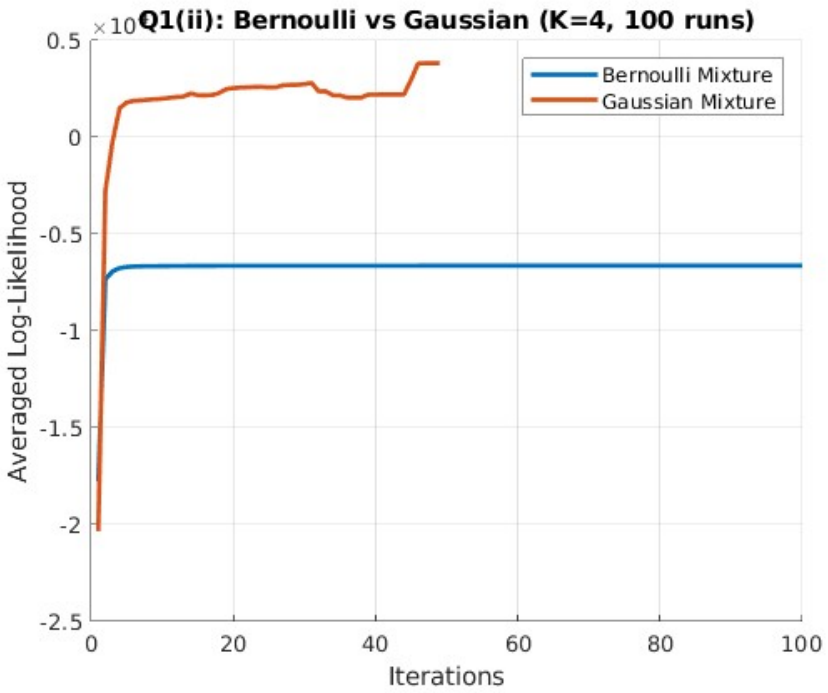

Figure 2: EM algorithm for GMM

This interpretation provides a probabilistic foundation for the K-Means algorithm and demonstrates that it is a special case of the EM procedure.

We assume that the data points $\{x_1, x_2, \ldots, x_N\}$ are generated from a mixture of K spherical Gaussian distributions with equal mixing coefficients and equal variances. The probability density of the mixture model is:

$$p(x_n) = \sum_{k=1}^{K} \pi_k \mathcal{N}(x_n \mid \mu_k, \sigma^2 I)$$

In the limit $\sigma^2 \to 0$, each data point is effectively assigned to the nearest mean, corresponding to a hard cluster assignment.

E-Step

For the general EM algorithm on GMMs, the posterior responsibility is:

$$r_{nk} = \frac{\pi_k \mathcal{N}(x_n \mid \mu_k, \sigma^2 I)}{\sum_{j=1}^{K} \pi_j \mathcal{N}(x_n \mid \mu_j, \sigma^2 I)}$$

As $\sigma^2 \to 0$, the Gaussian with the smallest squared distance $\|x_n - \mu_k\|^2$ dominates the numerator. Hence, the responsibility becomes:

$$r_{nk} = \begin{cases} 1, & \text{if } k = \arg\min_j \|x_n - \mu_j\|^2 \\ 0, & \text{otherwise} \end{cases}$$

This corresponds to assigning each data point to the closest cluster center.

M-Step

Given the hard cluster assignments r_{nk}, the mean of each cluster is updated as:

$$\mu_k = \frac{\sum_{n=1}^{N} r_{nk}\, x_n}{\sum_{n=1}^{N} r_{nk}}$$

This update moves each cluster center to the average of all data points currently assigned to it. Since $\pi_k = 1/K$ and $\Sigma_k = \sigma^2 I$ are fixed, only the means are updated.

Objective Function

The K-Means algorithm minimizes the distortion function (within-cluster sum of squares):

$$J = \sum_{n=1}^{N} \sum_{k=1}^{K} r_{nk} \|x_n - \mu_k\|^2$$

Each iteration of the algorithm decreases this objective until convergence.

Thus, K-Means can be interpreted as a limiting case of the EM algorithm applied to a GMM under equal isotropic covariances and uniform priors. The E-step corresponds to hard assignments, and the M-step corresponds to updating cluster means. This connection provides an elegant probabilistic justification for K-Means.

The K-Means algorithm is run 100 times with K=4, and the averaged objective function (sum of squared distances) is plotted against the number of iterations in Fig. 3.

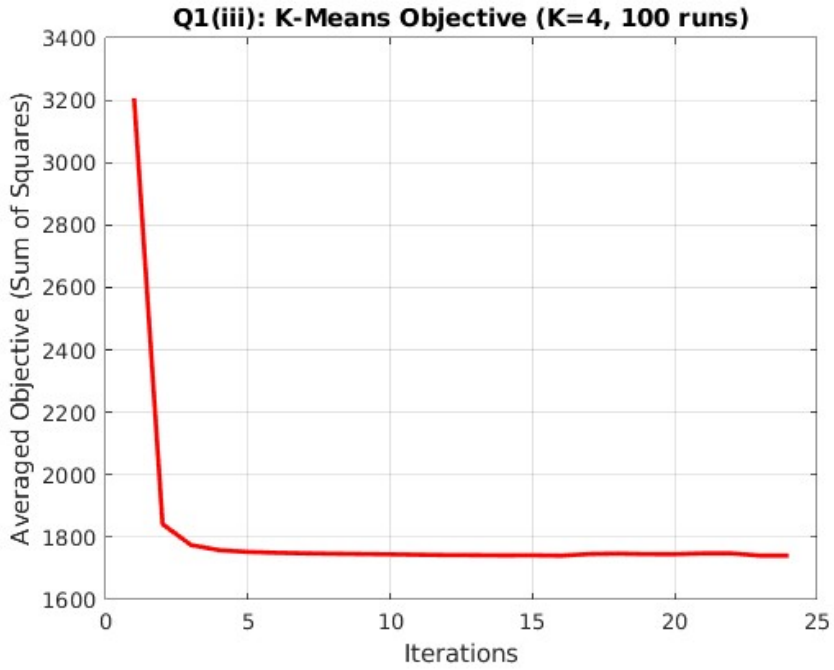

Figure 3: K-Means Objective vs. Iterations

The plot shows that the K-Means objective function converges extremely quickly, stabilizing in fewer than 10 iterations. The objective (distortion) drops from an average initial value of approx. 3200 to a stable minimum of approx. 1750. This rapid convergence indicates that the clusters are well-defined and easily separable in terms of Euclidean distance.

(iv) **Among the three different algorithms implemented above, which do you think you would choose to for this dataset and why?**
Sol: Based on the data's properties and the experimental results, the

Mixture of Bernoulli Distributions (BMM) is the most appropriate algorithm for this dataset.

- **Gaussian Mixture Model (GMM):** This is the **worst choice**. It is fundamentally and theoretically inappropriate. The GMM is designed for continuous data, but our dataset is strictly binary ($\{0, 1\}^{50}$). Treating this binary data as continuous is a model mismatch. The resulting positive log-likelihood (Fig. 2) is a misleading numerical artifact of a Probability Density Function (which can exceed 1) and is not comparable to the Bernoulli's log-likelihood.
- **K-Means:** This is a **reasonable but suboptimal choice**. As seen in Fig. 3, it converges well and finds stable cluster centers. Its objective is to minimize Euclidean distance, which is a valid measure of similarity. However, it is not a probabilistic model and only provides cluster centers (centroids).
- **Bernoulli Mixture Model (BMM):** This is the **best and most principled choice**.
  (a) **Correct Model Assumption:** It is the only one of the three algorithms that is statistically designed for the data type (a mixture of binary data).
  (b) **Interpretability:** It provides a much richer output than K-Means. Instead of just a centroid, the BMM gives a parameter vector $\boldsymbol{\mu}_k$ for each cluster, where each element μ_{kj} represents the probability that the j-th feature is '1' for that cluster. This is highly interpretable.
  (c) **Proven Fit:** The fast and stable convergence of its log-likelihood (Fig. 1) shows it fits the data structure effectively, just as K-Means does, but in a way that is statistically sound.

Therefore, for its theoretical correctness, strong performance, and interpretable probabilistic results, the Bernoulli Mixture Model is the clear choice.

Q2:

Refer to **Q2.m** file. **You are given a data-set in the file A2Q2Data_train.csv with 10000 points in** $(\mathbb{R}^{100}, \mathbb{R})$ **(Each row corresponds to a datapoint where the first 100 components are features and the last component is the associated y value).**

(i) **Obtain the least squares solution** w_{ML} **to the regression problem using the analytical solution.**
Sol: The goal of the least-squares problem is to find a weight vector $\mathbf{w}$ that minimizes the sum of squared differences between the predicted values $\mathbf{Xw}$ and the true values $\mathbf{y}$.

First, an intercept (bias) term is added to the feature matrix. The $N \times D$ data matrix $\mathbf{X}$ is augmented to become $\mathbf{X'} \in \mathbb{R}^{N \times (D+1)}$ by prepending a column of ones.

The objective function $J(\mathbf{w})$ to minimize is:

$$J(\mathbf{w}) = \frac{1}{2}||\mathbf{X'w} - \mathbf{y}||_2^2 = \frac{1}{2}(\mathbf{X'w} - \mathbf{y})^T(\mathbf{X'w} - \mathbf{y})$$

To find the minimum, we set the gradient with respect to $\mathbf{w}$ to zero:

$$\nabla_{\mathbf{w}} J(\mathbf{w}) = (\mathbf{X'})^T(\mathbf{X'w} - \mathbf{y}) = 0$$

This leads to the **Normal Equations**:

$$(\mathbf{X'})^T\mathbf{X'w} = (\mathbf{X'})^T\mathbf{y}$$

The analytical solution, also known as the Maximum Likelihood (ML) solution w_{ML}, is found by solving for $\mathbf{w}$:

$$\mathbf{w}_{ML} = ((\mathbf{X'})^T\mathbf{X'})^{-1}(\mathbf{X'})^T\mathbf{y}$$

This $\mathbf{w}_{ML}$ vector serves as the "ground truth" optimal solution for benchmarking the iterative algorithms in the next two parts.

(ii) **Code the gradient descent algorithm with suitable step size to solve the least squares algorithms and plot** $||w^t - w_{ML}||_2$ **as a function of t. What do you observe?**
Sol: Algorithm: Batch Gradient Descent (GD) is an iterative algorithm that updates the weights by taking steps in the negative direction of the gradient computed over the *entire* dataset. The update rule is:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \alpha \nabla J(\mathbf{w}^{(t)})$$

where α is the step size and $\nabla J(\mathbf{w}) = \mathbf{X'}^T(\mathbf{X'w} - \mathbf{y})$.

Step Size: A stable step size α can be chosen as $\alpha < 1/L$, where L is the Lipschitz constant of the gradient, given by the maximum eigenvalue

λ_{max} of the Hessian $\nabla^2 J(\mathbf{w}) = \mathbf{X'}^T\mathbf{X'}$. We set $\alpha = 1/(1.1 \cdot \lambda_{max}(\mathbf{X'}^T\mathbf{X'}))$ to ensure stable convergence.

Plot & Observations: The plot for this part is combined with part (iii) in Fig. 4. The solid blue line shows the L2-norm of the error, $||w^{(t)} - w_{ML}||_2$, versus the iteration count on a log-scale.

Observation: The plot shows a smooth, linear descent on the log-scale, which corresponds to an exponential (geometric) convergence rate. The error decreases steadily and rapidly, converging towards machine precision. This demonstrates the classic, stable convergence behavior of batch GD with a well-chosen step size on a convex problem.

(iii) **Code the stochastic gradient descent algorithm using batch size of 100 and plot $||w^t - w_{ML}||_2$ as a function of t. What are your observations?**
Sol: Stochastic Gradient Descent (SGD) estimates the full gradient using only a small "mini-batch" of data at each step. For a mini-batch $\mathcal{B}$ of size 100, the stochastic gradient is $\nabla J_{\mathcal{B}}(\mathbf{w}) = \mathbf{X}'^T_{\mathcal{B}}(\mathbf{X}'_{\mathcal{B}}\mathbf{w} - \mathbf{y}_{\mathcal{B}})$.

The data was shuffled at the beginning of each epoch, and a decaying learning rate $\eta_t = \eta_0/\sqrt{e}$ was used (where e is the epoch number) to help the algorithm settle near the minimum.

Plot & Observations: The plot in Fig. 4 (red dashed line) shows the error $||w^{(t)} - w_{ML}||_2$ for SGD.
Observation: The error for SGD also decreases, but at a **much slower rate per iteration** compared to batch GD. Furthermore, the curve is clearly flattening out at a much higher error "floor" (around $0.4 - 0.5$), while batch GD continues to converge to a far more precise solution.

This illustrates the key trade-off of SGD:

- Each step is less accurate because it only uses a small sample (100 points) of the data, so it makes less progress per iteration.
- The inherent randomness of the mini-batch gradient prevents the algorithm from ever converging to the exact minimum, causing it to "bounce around" in a neighborhood of the solution, which results in the higher error floor.

While slower per iteration, SGD is typically much faster in terms of wall-clock time, as one batch GD iteration requires processing all 10,000 points, while an SGD iteration only processes 100.

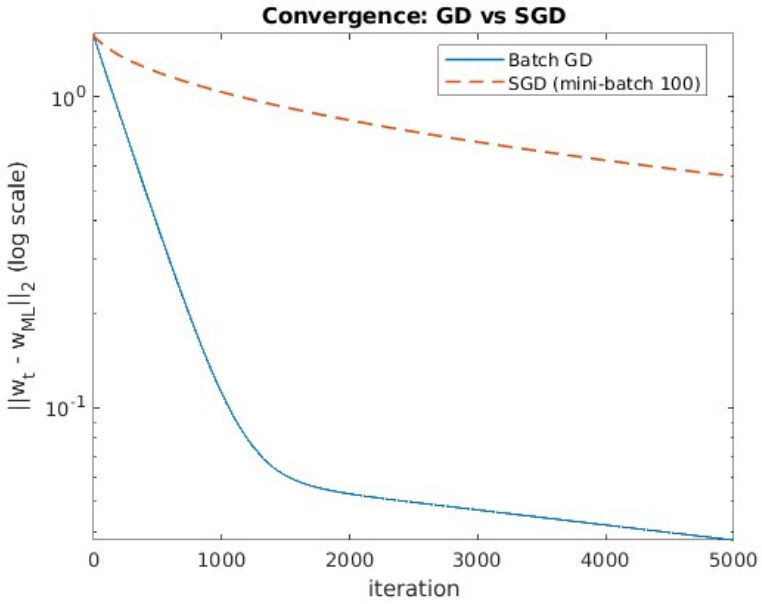

Figure 4: Convergence comparison of Batch Gradient Descent (GD) and Stochastic Gradient Descent (SGD) with batch size 100. The error $||w_t - w_{ML}||_2$ is plotted on a log scale.

(iv) **Code the gradient descent algorithm for ridge regression. Cross-validate for various choices of λ and plot the error in the validation set as a function of λ. For the best λ chosen, obtain w_R. Compare the test error (for the test data in the file A2Q2Data_test.csv) of w_R with w_{ML}. Which is better and why?**
Sol: Algorithm (Ridge Regression): Ridge Regression adds an L_2 penalty term to the least-squares objective to prevent overfitting. The new objective is:

$$J_{\text{Ridge}}(\mathbf{w}) = \frac{1}{2}||\mathbf{X}'\mathbf{w} - \mathbf{y}||_2^2 + \frac{\lambda}{2}||\mathbf{w}||_2^2$$

The analytical solution for this is:

$$\mathbf{w}_R = ((\mathbf{X}')^T\mathbf{X}' + \lambda\mathbf{I})^{-1}(\mathbf{X}')^T\mathbf{y}$$

Cross-Validation: To find the optimal regularization strength λ, 5-fold cross-validation was performed. The training data was split into 5 folds. For each of 50 candidate λ values (from 10^{-6} to 10^3), the model was trained on 4 folds and the mean squared error (MSE) was computed on the 5th (validation) fold. The validation errors were averaged across all 5 folds.

Plot & Results: The plot in Fig. 5 shows the average validation MSE as a function of λ. The error is relatively flat for $\lambda < 10^0$, and then rises sharply, indicating underfitting as the penalty becomes too strong.

The best value, λ_{best}, was found by cross-validation to be **2.683**.

This optimal λ_{best} was used to train the final Ridge model $\mathbf{w}_R$ on the *entire* training dataset.

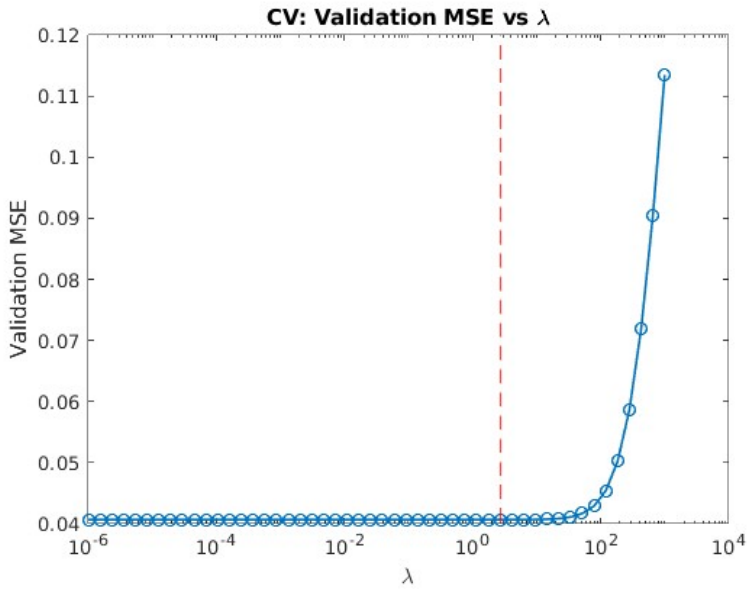

Figure 5: 5-fold cross-validation error (MSE) as a function of the regularization parameter λ. The optimal $\lambda = 2.683$ is marked by the red dashed line.

Coefficient Shrinkage Path: To visualize the effect of the L_2 penalty, we can plot the magnitude of each coefficient w_j (excluding the intercept) as a function of λ. This is known as the regularization path.

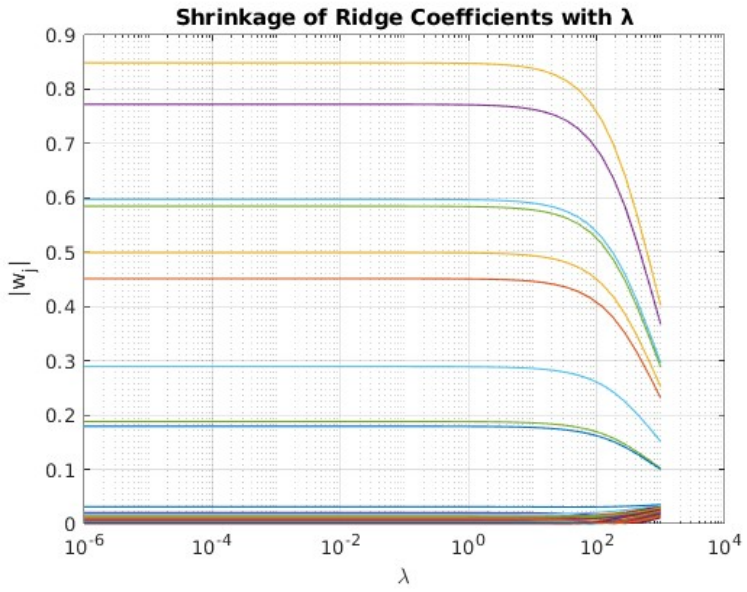

Figure 6: Regularization path for Ridge coefficients. As λ increases, the magnitudes of all coefficients are shrunk towards zero.

Observation: As seen in Fig. 6, when λ is very small (left side), the coefficients have large magnitudes, corresponding to the complex w_{ML} model. As the penalty λ increases (moving right), all coefficients are smoothly and uniformly "shrunk" towards zero. This shrinkage is what reduces the model's variance and prevents overfitting.

Test Error Comparison: Test Error Comparison: Finally, the performance of the w_{ML} (unregularized) and w_R (regularized) models was compared on the unseen test data. The results were:

- **Least Squares (ML) Test MSE: 0.370752**
- **Ridge (R) Test MSE: 0.369626**

Conclusion: Which is better and why?

The **Ridge Regression model ($\mathbf{w}_R$) is better**, as shown by its lower Test MSE ($0.369626 < 0.370752$).

The reason is that the original problem has 101 features (including intercept). With so many features, the unregularized w_{ML} model **overfits** to the training data. It learns complex patterns, including noise, that are specific to the training set but do not generalize to the test set.

The λ penalty in Ridge Regression forces the model's weights to be smaller, preventing any single feature from having too much influence. This "shrinks"

the weights, creating a simpler model that **generalizes better** to new, unseen data, resulting in a lower test error.